

LiftLog (Workout Tracker App) Project Spec

HCI 584 – Nathan Hines – [Github Repo](#)

General Description

LiftLog is a personal strength-training tracker application. The user records each workout (which exercises they did, and for every set the reps and the weight used) and the app keeps that history in a simple file on their computer. Over weeks and months that history becomes a personal training record the user can look back on, ask questions of, and chart. The whole point is to replace the scattered note-taking that lifters usually do (a phone Notes app, a paper notebook) with something structured enough to show/track progress.

The real value of the app is the analysis and visualization layer that sits on top of the log. Once the data is being captured cleanly, the user can pick any single exercise and see how their top working weight has trended over time, how much total work (volume) they did in each session, and whether they have hit any new personal records (PRs). The app detects PRs automatically from the log rather than asking the user to track them by hand.

Because all the data is generated by the user, the app does not depend on any external API or internet connection. This keeps it simple, private, and entirely self-contained. The only “data source” is the user’s own workout log file, plus a small built-in library of exercises (with category, movement pattern, and equipment) that the logger uses to keep exercise names consistent and to power future features.

The project relies on a small set of established Python libraries. Pandas for loading, querying, and saving the data, and matplotlib (and/or plotly) for the charts. Data persists as a plain CSV file read into a pandas DataFrame and written back out.

Version 1 can be a set of .py files / a CLI script that handle the data flow for logging a workout, appending it to the CSV, and loading it back. Also a Jupyter notebook used during this Version to build and refine the visualizations against the saved data before any of that logic gets wired into a real interface. For a future GUI I’m leaning towards web app using Streamlit.

Task Vignettes (User Flows)

Vignette 1: Logging a workout

Nathan has been lifting for three months but has only been keeping notes in his phone. After today’s session he opens LiftLog and chooses “Log a workout.” It defaults to today’s date. He adds his first exercise by picking “Back Squat” from the exercise library, then enters each set as he actually did it: 185 lbs for 5 reps, again for 5, again for 5, and a final heavy set of 3 at 190. He adds Bench Press and Deadlift the same way, types a quick note on the deadlift (“hard final set”), and saves. The app appends the new rows to his workout log file and confirms it saved.

Technical details

- Date defaults to today but is editable so missed sessions can be back-filled.

- The user selects exercises from the built-in exercise_library (a list of dicts) so names stay consistent for later filtering/charting.
- Each set is stored as its own row. The logger loops: “add another set? Y/N” / “add another exercise? Y/N” until the user is done.
- New rows are appended to the existing DataFrame and written back to workout_log.csv. The file is created automatically if it does not exist yet.
- Input is validated: weight and reps must be non-negative numbers; an unknown exercise name prompts a confirm-or-reselect.

Vignette 2: Viewing progress for one exercise

A week later Nathan wants to know if his squat is actually moving. He opens LiftLog, chooses “View progress,” and selects “Back Squat.” The app pulls every Back Squat set he has ever logged, groups them by date, and shows a line chart of his top set per session climbing from 135 lbs up to 190 over ten sessions. Below the chart it lists his session-by-session volume so he can see not just whether the weight went up but whether he is doing more total work.

Technical details

- Filter the DataFrame to one exercise_name, then group by date.
- Two derived series per session: top set = max(weight) on that date, and session volume = sum(reps × weight) across that day’s rows.
- Chart is rendered with matplotlib (developed first in the notebook). X-axis = date, Y-axis = weight; volume can be a second line or a separate plot.
- In V1 (CLI), the chart opens in a window or is saved as a .png; in a future Streamlit GUI it renders inline.
- Edge cases: an exercise with only one logged session (single point, no trend) and an exercise never logged (friendly “no data yet” message).

Vignette 3: Spotting a personal record

On his next heavy day Nathan pulls 230 lbs on the deadlift for the first time. He logs the session as usual. When he checks his summary, the app flags it: “New Deadlift PR: 230 lbs (previous best 225).” He did not have to track that himself, the app compared the new lift against everything in the log and noticed it was a record.

Technical details

- A PR for an exercise = max(weight) ever recorded for that exercise_name across the whole log.
- On save (or on demand), compare each exercise’s new max against its prior max; if it increased, surface a PR message.
- A simple summary view lists current PRs per exercise and recent activity (last N sessions).
- Optional/stretch: log the PR’s date so PRs can themselves be charted over time.

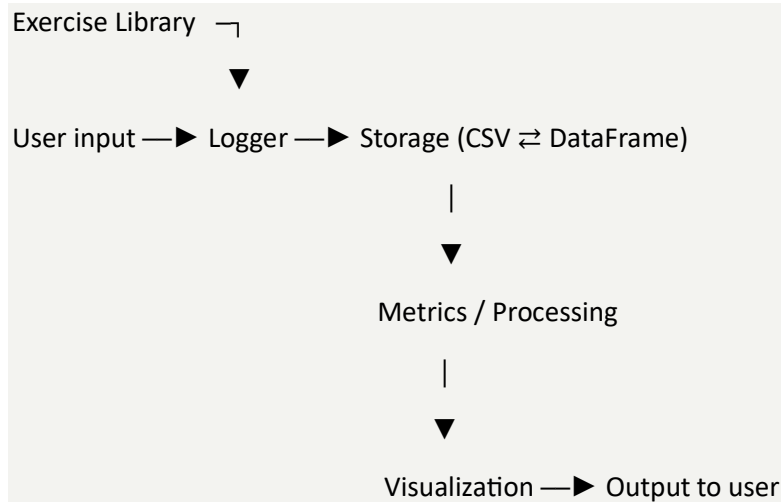
Technical Flows

Blocks and data flow

The arrows show what data each block passes to the next. Data types are noted in brackets.

- **Exercise Library** supplies valid exercise names + metadata (category, movement_pattern, equipment, muscle_group) to the Logger and to future filtering.

- **Workout Logger / Input** [user keyboard input → list of new row dicts] → collects date, exercise, and each set; hands new rows to Storage.
- **Storage (load / save)** [CSV ↔ pandas DataFrame] → reads workout_log.csv into a DataFrame on start, appends new rows, writes back. The single source of truth.
- **Metrics / Processing** [DataFrame → smaller DataFrames / numbers] → filters by exercise, groups by date, computes top set, session volume, average reps, and PRs.
- **Visualization** [processed DataFrame → chart / .png] → turns the metric series into line charts (matplotlib; developed in the notebook).
- **Output / Display** [text + charts → user] → prints summaries and PR flags to the console and shows/saves charts. Becomes the Streamlit page later.



Exercise Library Schema (exercise_library)

The library is a list of dictionaries, each with five flat fields:

```

exercise_library = [
    {"name": "Back Squat", "category": "Strength",
     "movement_pattern": "Squat", "equipment": "Barbell",
     "muscle_group": "Quads"},
    {"name": "Deadlift", "category": "Strength",
     "movement_pattern": "Hinge", "equipment": "Barbell",
     "muscle_group": "Hamstrings"},
    {"name": "Bench Press", "category": "Strength",
     "movement_pattern": "Horizontal Push", "equipment": "Barbell",
     "muscle_group": "Chest"},
]

```

Field	Purpose
name	Written into the CSV exercise_name column; the user picks this from the list
category	Strength vs. Bodyweight — helps handle the weight = 0 case in metrics
movement_pattern	Squat, Hinge, Horizontal Push, etc. — powers “what can I do instead?” substitution
equipment	Barbell, Dumbbell, Cable, Machine, Bodyweight — powers equipment-based filtering
muscle_group	Quads, Chest, Biceps, etc. — the single primary muscle the exercise trains

CSV schema (workout_log.csv):

```
date,exercise_name,set_number, reps, weight, notes
2026-06-03,Back Squat,1,5,185,Felt strong
2026-06-03,Back Squat,2,5,185,
2026-06-03,Back Squat,3,5,185,
2026-06-03,Back Squat,4,3,190,grinder
2026-06-03,Bench Press,1,8,135,Good speed
2026-06-03,Deadlift,1,5,225,Hard final set
```

The CSV data model approach is one row per set. Each set is a record; a “workout” is simply all the rows sharing a date, and an “exercise within a workout” is all the rows sharing that date and exercise name.

Field	Type	Notes
date	date / string (YYYY-MM-DD)	Groups rows into a session
exercise_name	string	Should match the exercise library
set_number	int	1, 2, 3... within that exercise that day
reps	int	Reps completed in the set
weight	float	Load in lbs (0 for bodyweight)
notes	string (optional)	Free text, may be blank

Metric definitions

Metric	Definition
Top set (day)	Highest weight used for that exercise on that date — $\max(\text{weight})$
Set volume	$\text{reps} \times \text{weight}$ for a single set
Session volume	Sum of $(\text{reps} \times \text{weight})$ across all of that exercise's sets that day
PR	Highest weight ever logged for that exercise — $\max(\text{weight})$ across the whole log
Average reps	$\text{Total reps} \div \text{number of sets}$

Final Assessment

Biggest / most unexpected change from the sketch

The data model. The sketch stored a whole exercise on one line with a single "sets" and as I worked through the spec it was made it clear that real sets aren't identical, so I switched to one row per set with a `set_number`. A second, smaller call was deciding how richly to tag the exercise library: I considered full primary/secondary muscle lists but pulled back to a single `muscle_group` per exercise to keep V1 simple, leaving multi-muscle tagging as a stretch goal.

Confidence in implementing the spec as written

Fairly high for the core. Logging to a CSV, loading it with pandas, filtering, grouping, and the PR/volume math are all things I have worked with in this class and last semesters class. The visualizations I want to prototype in a notebook first to get them right before any UI work.

Biggest problem I NEED to solve

Getting the data model and the save/append cycle solid. Ultimately, if rows aren't written cleanly and consistently, every chart and metric downstream breaks. Validating input (so bad reps/weight never enter the file) is part of this.

Parts I'm least familiar with / may need help on

Moving from the CLI + notebook into an actual Streamlit interface in a later version, and the structure of the eventual real-time logging flow. Guidance on the cleanest way to wire the existing data functions into Streamlit would help most.